Deep Reinforcement Learning

JENNIFER JOYCE M. MONTEMAYOR - MAULANA

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

What is Deep Reinforcement Learning?

Deep Reinforcement learning combines **Reinforcement Learning (RL)** and **Deep Learning (DL)**.

Goal:

> To create an **agent** that learns optimal behavior (presented through a sequence of decisions) by interacting with an **environment**, similar to how humans and animals learn via **trial and error**.

What is Deep Reinforcement Learning?

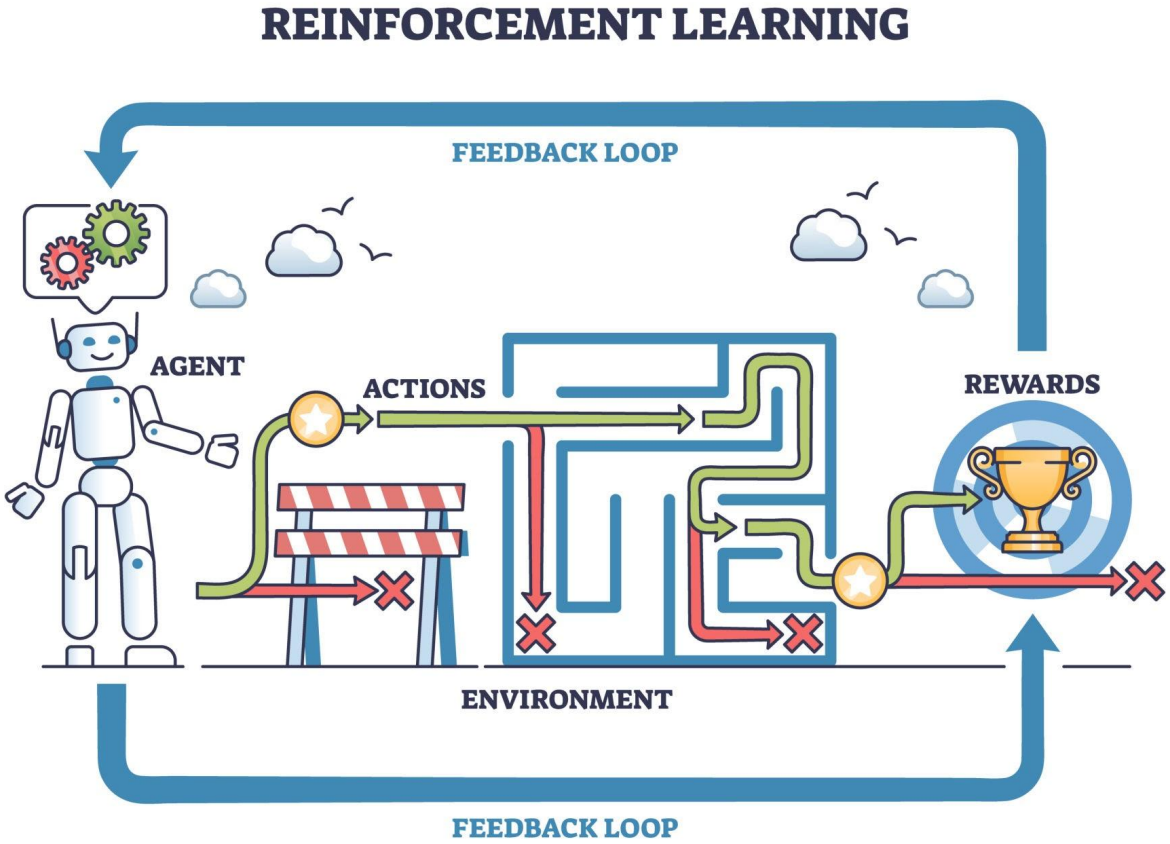
Deep Reinforcement learning combines **Reinforcement Learning (RL)** and **Deep Learning (DL)**.

Goal:

> To create an **agent** that learns optimal behavior (presented through a sequence of decisions) by interacting with an **environment**, similar to how humans and animals learn via **trial and error**.

The pillars of Machine Learning:

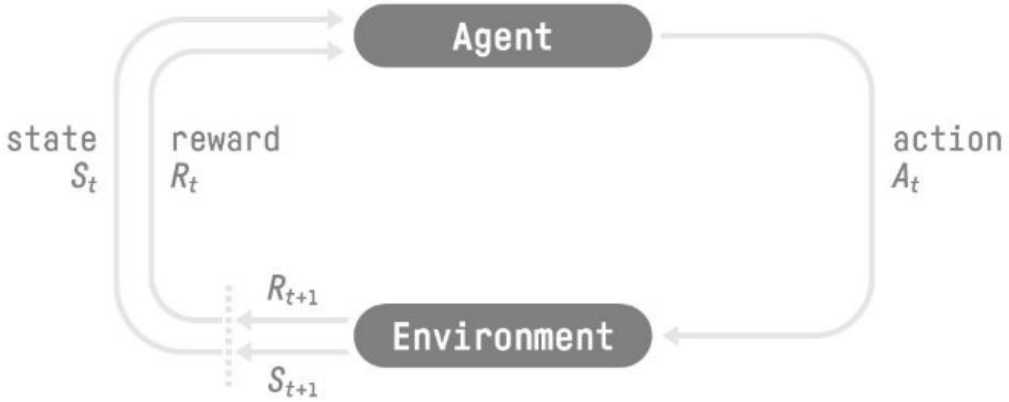
- Supervised Learning: Learning from labeled data
- Unsupervised Learning: Learning patterns from unlabeled data
- Reinforcement Learning: Learning from interaction and feedback (rewards).

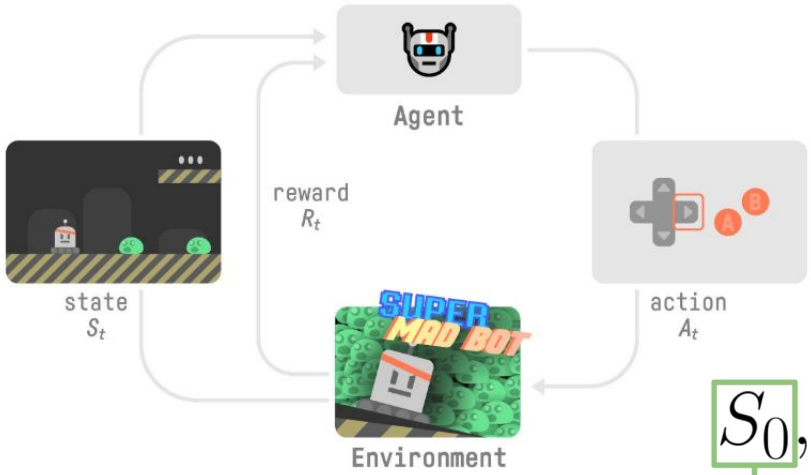


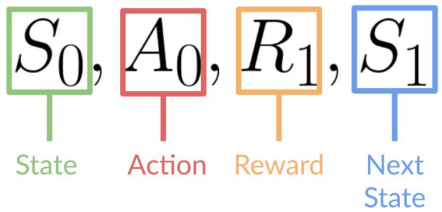
The entire Reinforcement Learning process is modeled as an Agent interacting with an Environment in a sequential decision-making loop.

The Agent-Environment Interaction

- State (S_t): A complete description of the environment at time t. The agent uses the state to decide what to do next.
- Action (A_t): The choice made by the agent at time t. This action influences the environment.
- Reward(R_{t+1}): A scalar feedback signal from the environment received after the action A_t is taken. This is the only signal guiding the agent's learning; it indicates the goodness of the last action.
- Transition Probabilities: Taking action A_t in state S_t leads to a new state S_{t+1} and yields a reward R_{t+1}. This cycle continues.







The entire Reinforcement Learning process is modeled as an Agent interacting with an Environment in a sequential decision-making loop.

The Agent-Environment Interaction

- State (S_t): A complete description of the environment at time t. The agent uses the state to decide what to do next.
- Action (A_t): The choice made by the agent at time t. This action influences the environment.
- Reward(R_{t+1}): A scalar feedback signal from the environment received after the action A_t is taken. This is the only signal guiding the agent's learning; it indicates the goodness of the last action.
- Transition Probabilities: Taking action A_t in state S_t leads to a new state S_{t+1} and yields a reward R_{t+1}. This cycle continues.

Markov Decision Process (MDP)

An MDP is the mathematical framework for RL. It formalizes the interaction between an agent and its environment.

It assumes the **Markov Property**:

"The future state of a system depends only on the current state and action, not on the entire history of past states and actions."

The ultimate goal of an agent is to maximize the total future reward, the Return G_t

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Where,

- Discount Factor(γ) $\in$ [0,1]. When close to 1, future rewards matter more making agent "far-sighted" focusing on long-term reward. When close to 0, focus is on immediate reward making agent "short-sighted".

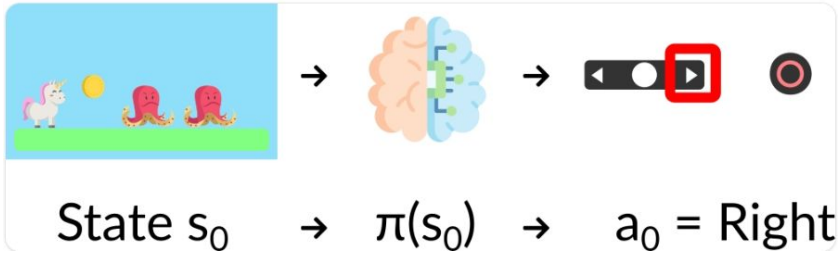
Core Reinforcement Learning Agent Components

- Policy (π - `Pi`)
  - The strategy of the agent - how does it map actions or choose actions based from observed states? What action to take given the state we are in?
  - Defines the behavior of the agent at a given time
  - Can either be,
    1. Deterministic policy: $\pi(s)=a$ picks the same action for a state
       Given state s, the policy always chooses the same action a
    2. Stochastic policy: $\pi(a|s)=P(A_t = a \mid S_t = s)$
       Policy outputs a probability distribution over actions, not a single action

       For a given state s, each action a has some probability,

       $$\pi(\text{Right}|s_5) = 0.7, \pi(\text{Up}|s_5) = 0.3$$

       Agent will choose Right 70% of the time, Up 30%





Core Reinforcement Learning Agent Components

- **Value Functions** (`V` and `Q`)
  - Predictions of future reward
  - Tell the agent how good a state or a state-action pair is
  - State-Value Function (`V^π(s)`): The expected return starting from state `s` and then following policy π

$$V^{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

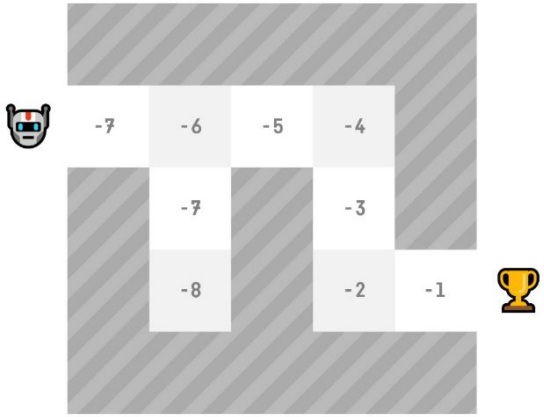
  - Action-Value Function (`Q^π(s, a)`): The expected return starting from state `s`, taking action `a`, and then following policy π

$$Q^{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

The value of a state is the expected discounted return the agent can get if it starts in that state, and then acts according to our policy.

$$v_{\pi}(s) = \mathbb{E}_{\pi}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right]$$

Value function — Expected discounted return — Starting at state s



*"The value function helps us **evaluate** and **improve** the policy:*
*we look at the values of states (or state–action pairs) which are **approximations of the expected cumulative reward** and then define a new policy that, in each state, picks actions leading to higher value."*

Core Reinforcement Learning Agent Components

- Model
  - The agent's representation of how the environment works
  - Internal model of the environment
  - Let the agent predict what next state and reward it will get for any state-action
  - Model-Based RL:
    - Transition model

      $P(s' \mid s, a)$

      probability that the next state is s' given current state s and action a

    - Reward model

      $R(s, a)$ (or $R(s, a, s')$)

      expected reward for taking action a in state s (possibly ending in s')
  - Model-Free RL:
    - Directly learns optimal policy or value function through interaction (trial and error)
    - Repeatedly collects (s, a, r, s')

      current state s, action taken in that state a, reward received after taking the action r, next state the agent ends up in s'

Markov Decision Process (MDP): Grid World

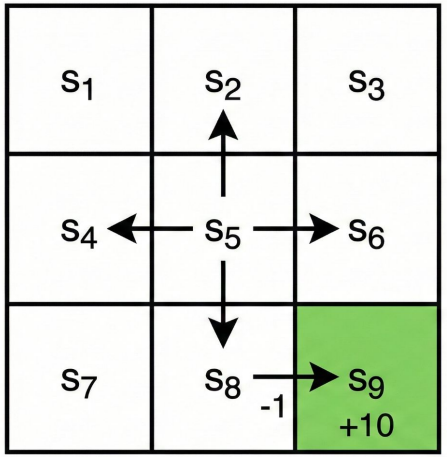
Scenario: An agent moves inside a 3x3 grid trying to reach a goal.

To “solve” this grid world we want a policy instead of a path.

1. Environment

Agent starts at a random square and wants to get to the “Goal” while avoiding “Walls”

Goal State = ?
Wall = ?
Standard Move = ?



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

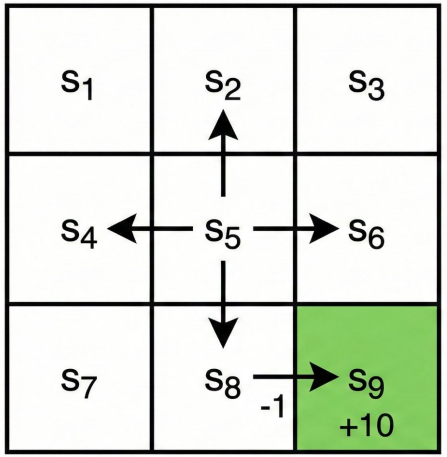
Discount (γ): 0.9

Markov Decision Process (MDP): Grid World

2. The Challenge: Stochasticity (The “Slip”)

Transition function P introduces uncertainty,

- 90% chance: Agent moves where you want
- 10% chance: Agent slips to a random adjacent cell



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

The agent cannot just plan the shortest route near a dangerous edge. If s_c is a cliff (very negative reward) instead of a safe square, the agent might prefer a longer, safer route through the center to avoid the 10% chance of slipping into disaster.

Markov Decision Process (MDP): Grid World

3. How the Agent “Learns”

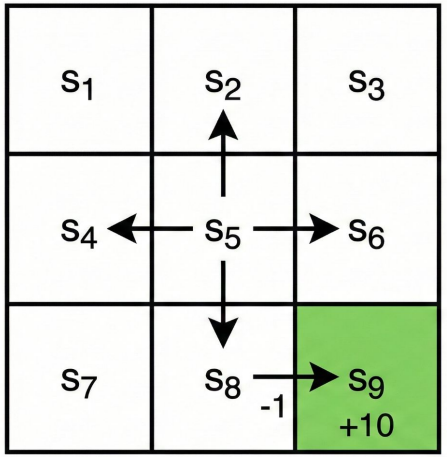
The agent calculates the Value (V) of every state.

$$\underbrace{V(s)}_{\text{The Score}} = \underbrace{\max_a \sum P(s'|s,a)[R+\gamma V(s')]}_{\text{The Bellman Equation}}$$

$$V^{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a)\left[r(s,a,s') + \gamma V^{\pi}(s')\right]$$

expectation over actions chosen by the policy

expectation over next states from the environment.



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

The agent will choose the action that gives the highest weighted average of the immediate reward plus the future value of the next state discounted by gamma.

Markov Decision Process (MDP): Grid World

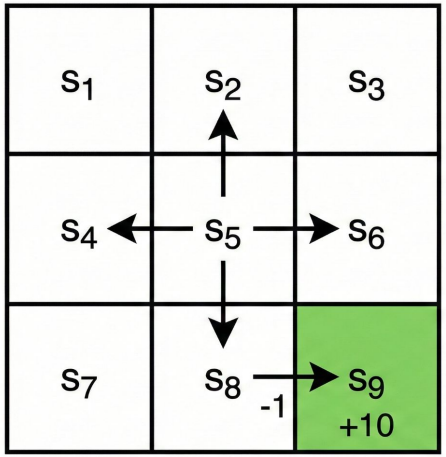
3. How the Agent “Learns”

The agent calculates the Value (`V`) of every state.

$$\underbrace{V(s)}_{\text{The Score}} = \underbrace{\max_a \sum P(s'|s,a)[R+\gamma V(s')]}_{\text{The Bellman Equation}}$$

Say for s_8, assuming we try to move Right toward the Goal s_9

```
V(s9) = 10 (Goal), V(s5) = 0 (Center)
Action: Right
Discount: 0.9
```



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

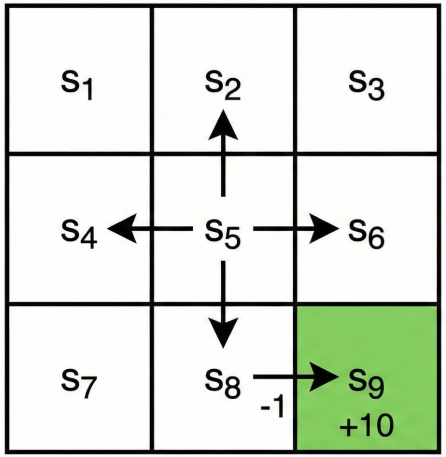
Agent calculates does not get the result of moving Right. It calculates for the expected return based on the probabilities.

Markov Decision Process (MDP): Grid World

3. How the Agent "Learns"

The agent calculates the Value (V) of every state.

$$\text{Expected Value} = \sum_{s'} \underbrace{P(s'|s,a)}_{\text{Weight}} \times \underbrace{[R + \gamma V(s')]}_{\text{Value}}$$



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

Say for s_8, assuming we try to move Right toward the Goal s_9

```
V(s9) = 10 (Goal), V(s5) = 0 (Center)
Action: Right
Discount: 0.9
```

Intended Outcome (90%): Successfully move to s_9

```
Reward: +10, Future Value: 10
0.9 x (10 + 0.9 x 0.0) = 9.0
```

Slip Outcome (10%): Slip to random adjacent e.g s_5

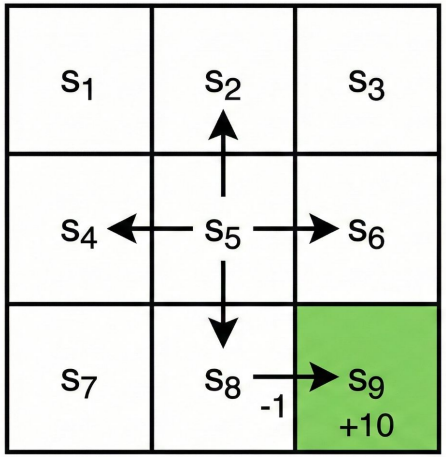
```
Reward: 0, Future Value: 0
0.1 x (0 + 0.9 x 0.0) = 0.0
```

Markov Decision Process (MDP): Grid World

3. How the Agent "Learns"

The agent calculates the Value (V) of every state.

$$\text{Expected Value} = \sum_{s'} \underbrace{P(s'|s,a)}_{\text{Weight}} \times \underbrace{[R + \gamma V(s')]}_{\text{Value}}$$



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

Say for s_8, assuming we try to move Right toward the Goal s_9

```
V(s9) = 10 (Goal), V(s5) = 0 (Center)
Action: Right
Discount: 0.9
```

```
Right : 9.0 + 0.0 = 9.0
```

Intended Outcome (90%): Successfully move to s_9

```
Reward: +10, Future Value: 10
0.9 x (10 + 0.9 x 0.0) = 9.0
```

Slip Outcome (10%): Slip to random adjacent e.g s_5

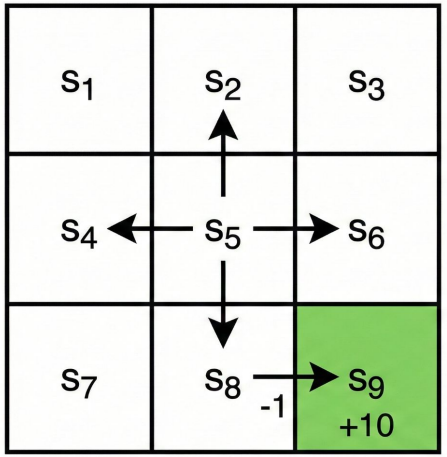
```
Reward: 0, Future Value: 0
0.1 x (0 + 0.9 x 0.0) = 0.0
```

Markov Decision Process (MDP): Grid World

4. An Optimal Policy π^*

A lookup table that can look like this,

$$\pi(s_1) = \text{Right}$$
$$\pi(s_2) = \text{Right}$$
$$\pi(s_3) = \text{Down}$$
$$\pi(s_8) = \text{Right}$$



Grid World MDP.

States (S): {s_1, …, s_9}

Actions (A): Up, Down, **Left, Right**

Transition (P): Intended 0.9, Slip 0.1

Reward (R): +10 Goal, -1 Wall, 0 Other

Discount (γ): 0.9

When the agent is on State 8, the rulebook says it must choose the action Right.

Why Deep RL?

Traditional Reinforcement Learning stores value for every state or state-action pair in a table.

Q-table where rows represents states (`s`) and columns represents actions (`a`). Each cell contains a Q-value: `Q(s,a)`.

```
Q(s,a) = expected future reward for taking action a in state s
```

Q-table informs the agent how good each action is in each state.

Suppose a robot is in a grid world with 4 states and has 2 actions: *left* and *right*.

State (s)	Left (a_1)	Right (a_2)
s_1	0.5	0.2
s_2	-0.1	0.8
s_3	0.0	0.0
s_4	1.0	-0.5

The complexity of learning grows exponentially with the number of possible states or actions.

Why Deep RL?







The complexity of learning grows exponentially with the number of possible states or actions.

Deep Reinforcement Learning (DRL) Agent

A DRL agent uses **Deep Neural Networks (DNNs)** to handle complex, high-dimensional inputs (e.g., images, sensor data) while learning how to act in an environment.

Deep Neural Networks serve as non-linear function approximators for the Policy and Value functions.

A. Policy Network
   - "actor" - choose and perform actions in the environment
   - takes the state S as input and outputs the probability of taking each action A that maximizes the expected return
   - updated using Policy Gradient methods

B. Value Network / Q-Network
   - "critic" - evaluates the actor's performance
   - evaluates the quality of a state or state-action pair
   - estimates expected future cumulative reward or the return
   - updated using Temporal Difference learning

Why Deep RL?







Deep reinforcement learning is used when:

- The state space is huge or high-dimensional (e.g., raw images, many sensors).
- We need to learn from trial-and-error interaction (no labeled dataset).
- Decisions are sequential and affect future states and rewards.

Common problem types:

1. **Robotics control & manipulation** (grasping, navigation, control from cameras).
2. **Games** (Atari from pixels, Go, complex video games).
3. **Autonomous driving** (lane keeping, path planning, control).
4. **Resource/traffic/network optimization** (data centers, IoT, scheduling).
5. **Healthcare treatment strategies** (sequential treatment decisions).

Deep Reinforcement Learning (DRL) Algorithms

Algorithm Family	Primary Objective	Network Structure	Key Advantage	Key Disadvantage	Example Algorithm
1. Value-Based	To learn the optimal Action-Value Function (Q*). The policy is derived (argmaxQ).	Typically a single Q-Network (DNN) that outputs Q-values for all discrete actions.	Highly sample efficient (Off-Policy), meaning it reuses data well.	Restricted to discrete action spaces. Prone to overestimation bias.	Deep Q-Network (DQN), Double DQN (DDQN)
2. Policy-Based	To learn the optimal Policy (π) directly.	A single Policy Network (DNN) that outputs action probabilities or parameters.	Naturally handles continuous action spaces and learns stochastic policies.	Very high variance in gradient estimates, leading to unstable training.	REINFORCE (Monte Carlo Policy Gradient)
3. Actor-Critic	To learn both the Policy (π) (Actor) and the Value Function (V) (Critic) collaboratively.	Two separate DNNs (Actor Network and Critic Network), often sharing base layers.	Combines the best of both: low variance (thanks to the Critic) and ability to handle continuous actions (thanks to the Actor).	Requires learning and tuning two networks simultaneously, which adds complexity.	PPO (Proximal Policy Optimization), DDPG, SAC

Deep Reinforcement Learning (DRL) Algorithms

Algorithm	Type	Core Contribution	Use Case / Environment
DQN	Value-Based	Introduced Experience Replay and Target Networks to stabilize training of Q-Learning with DNNs.	Atari Games (e.g., Breakout, Space Invaders) from raw pixel inputs.
PPO	Actor-Critic	Uses a clipped objective function to constrain policy updates, offering high stability and sample efficiency (for an on-policy method).	MuJoCo Robotics simulations (e.g., learning a humanoid to walk), Complex 3D navigation.
DDPG	Actor-Critic	Applied the DQN stability mechanisms (Experience Replay, Target Networks) to the Actor-Critic framework, enabling off-policy learning for continuous control.	Continuous Control Tasks like operating a robotic arm, vehicle trajectory following.
SAC	Actor-Critic	Maximizes a combination of reward and policy entropy, encouraging effective exploration and resulting in highly robust and stable continuous control policies.	State-of-the-art for continuous robotics and locomotion tasks.
REINFORCE	Policy-Based	The foundational policy gradient algorithm. Updates the policy based on the full episodic return (Gt).	Used mainly as a theoretical foundation today; rarely used alone due to high variance.

Resources

1. Reinforcement Learning: An Introduction
   By: Richard S. Sutton and Andrew G. Barto

2. MIT Deep Learning Lectures
   Deep reinforcement learning. https://introtodeeplearning.com/

3. Deep RL Course
   https://huggingface.co/learn/deep-rl-course/

4. Google Gemini
   Recent research and developments in deep reinforcement learning

5. ChatGPT (OpenAI)
   Interactive AI assistant for understanding core concepts of RL and DRL